

1 Preliminaries and Problem Setting

We begin by reviewing relevant background and formalizing the problem setting.

1.1 Regularized Quasi-Newton Method

A regularized quasi-Newton method is a variant of quasi-Newton methods that does not rely heavily on line-search techniques. This property is important for optimization in noisy evaluation environments. There has been renewed interest in regularized Newton-type methods, including cubic-regularization schemes [30, 33, 41], and we focus on quadratic regularization [9, 23, 25, 35, 38, 39, 46] for simplicity of analysis and implementation.

For $g_k := \nabla f(x_k)$ and an approximate Hessian $B_k \in \mathbb{R}^{n \times n}$, regularized quasi-Newton methods compute the step as

$$x_{k+1} = x_k + d_k, \quad d_k = -(B_k + \mu_k I)^{-1} g_k,$$

with a regularization parameter $\mu_k \geq 0$. Combining regularization with line search is also possible and has been explored [38]. Several strategies for choosing μ_k in nonconvex optimization have been proposed [35, 38, 39], and one of them is given by

$$\mu_k = c_1 \max(0, -\lambda_{\min}(B_k)) + c_2 \|\nabla f(x_k)\|^\delta,$$

where $c_1 > 1$, $c_2 > 0$, $\delta \geq 0$, and $\lambda_{\min}(B_k)$ denotes the smallest eigenvalue of B_k . This choice guarantees that $B_k + \mu_k I$ is positive definite, ensuring that d_k is a descent direction. We adapt such regularization strategies to noisy evaluation environments.

1.2 Objective-Function-Free Optimization

Objective-Function-Free Optimization (OFFO) refers to a class of methods that avoid explicit evaluations of the objective function and rely solely on gradient information [18, 19]. Such methods are particularly attractive when function values are unreliable due to noise. Similar adaptive trust-region approaches are also discussed [16].

Prominent examples of OFFO include adaptive gradient methods such as AdaGrad [11] and AdaGrad-Norm [42]. Their update rules are

$$\text{(AdaGrad)} \quad x_{k+1}^{(i)} = x_k^{(i)} - \frac{1}{\theta \sqrt{\varsigma + \sum_{j=0}^k (g_j^{(i)})^2}} g_k^{(i)}, \quad (i = 1, 2, \dots, n)$$

$$\text{(AdaGrad-Norm)} \quad x_{k+1} = x_k - \frac{1}{\theta \sqrt{\varsigma + \sum_{j=0}^k \|g_j\|^2}} g_k$$

respectively, where $\varsigma > 0$ and $\theta > 0$ are algorithmic parameters, and $x^{(i)}$ denotes the i -th component of a vector x .

1.3 Problem Setting

We consider optimization problems in which exact objective function values are unavailable, following the recent literature on error-aware optimization [2, 6, 7, 17, 34, 37]. When both objective values and gradients are heavily corrupted by noise, reliable optimization cannot be guaranteed. Thus, meaningful optimization requires that at least one source of information be sufficiently accurate. This work focuses on problems arising from finite-precision computations or inexact evaluations, where objective function evaluations are subject to non-negligible errors while gradient information can be computed with relatively high accuracy. **Settings dominated by gradient errors, such as stochastic approximation and inexact-gradient methods** [3, 36, 43], are beyond the scope of this work.

In the following, we formalize the assumptions on function value and gradient evaluations. We assume that only an error-contaminated function value $\bar{f}(x)$ is accessible, satisfying the hybrid absolute-relative error model [21, 22]:

$$|f(x) - \bar{f}(x)| \leq \varepsilon_f \max(1, |f(x)|), \quad (1)$$

where $0 \leq \varepsilon_f < 1$ is an error rate. The error is predominantly relative when $|f(x)|$ is large and effectively absolute when $|f(x)|$ is small, capturing the typical behavior of rounding errors in IEEE 754 floating-point arithmetic [21]. **The value of ε_f serves as a safety margin for errors in the computed function values. Underestimating ε_f can violate eq. (1) and cause the method to treat unreliable function-value comparisons as informative. By contrast, choosing a larger value preserves the analysis as long as $\varepsilon_f < 1$ and the error bound remains valid, although it makes the method rely earlier or more frequently on the more conservative cumulative-gradient scaling. Because a function evaluation generally involves many floating-point operations whose errors can accumulate, we choose a precision-dependent multiple of machine epsilon as an aggregate error allowance rather than identifying ε_f with the error from a single rounding operation. Specifically, our choices are guided by numerical stopping rules such as the default SciPy L-BFGS-B function tolerance, which uses 10^7 times machine epsilon [40]. The values used at each precision and a sensitivity study with under- and overestimated values are reported in section 5.**

We next specify the assumption on gradient evaluations. **Let $g(x)$ be the gradient returned by the available gradient oracle at x . In practice, termination is typically based on a gradient tolerance $\varepsilon_{\text{gtol}}$, and $\|g(x)\| \leq \varepsilon_{\text{gtol}}$ is used as a stopping criterion. Thus, before termination, $\|g(x)\| > \varepsilon_{\text{gtol}}$. If the gradient error $\|g(x) - \nabla f(x)\|$ were comparable to or larger than $\varepsilon_{\text{gtol}}$, this stopping criterion would not reliably indicate stationarity even when $\nabla f(x) = 0$. Meaningful termination therefore requires the gradient error to be sufficiently small relative to $\varepsilon_{\text{gtol}}$. In particular, before termination, we have in mind the regime $\|g(x) - \nabla f(x)\| \ll \varepsilon_{\text{gtol}} < \|g(x)\|$. Under this regime, $g(x)$ is a sufficiently accurate approximation of $\nabla f(x)$ for the required accuracy. For simplicity, the convergence analysis adopts the following exact-gradient assumption.**

Assumption 1 (Exact gradient) *The algorithm has access to a gradient oracle g such that, for every evaluated point x ,*

$$g(x) = \nabla f(x). \quad (2)$$

This assumption is a modeling simplification for the analysis rather than a claim that practical gradients are exact. Notably, unlike a Wolfe line search, the proposed step-acceptance condition does not require a gradient evaluated at the trial point. The available gradient is nevertheless used both to form the search direction and in the acceptance comparison, so this observation alone does not establish robustness to gradient errors. The function-value-only experiments satisfy Assumption 1, whereas the joint-noise experiments deliberately violate it and lie outside the theory. The latter experiments are included to assess the empirical robustness of the proposed method, not to validate or extend the convergence result.

2 Proposed Algorithm

We propose a new algorithm for the problem setting stated in section 1.3. We draw inspiration from the OFFO framework to design robust regularization and scaling strategies. Unlike purely OFFO methods, our algorithm exploits function values when they are numerically reliable and smoothly switches to gradient-based control otherwise. A regularized quasi-Newton direction alone does not specify how to update the regularization scale once comparisons of noisy function values become uninformative, whereas the cumulative gradient scale provides such an update without further objective comparisons. Using that scale at every iteration would instead discard useful function information and may be unnecessarily conservative, which motivates switching between unregularized steps supported by function comparisons and regularized steps based on the cumulative gradient scale. This hybrid behavior is intended to balance stability under unreliable function values with the efficiency of unregularized quasi-Newton steps when function information is useful. The pseudocode of our algorithm is given in Algorithm 1. The minimum of the empty set is defined as $+\infty$ in line 3. Algorithm 2 is an auxiliary line search algorithm called in Algorithm 1. In the following, we explain the three main components of Algorithm 1.

Computation of the Regularized Quasi-Newton Direction (line 9): The first component is the computation of the regularized quasi-Newton direction d_k stated in section 1.1. Given $g_k = g(x_k) = \nabla f(x_k)$, an approximate Hessian B_k , and a regularization parameter μ_k , we compute the search direction $d_k = -(B_k + \mu_k I)^{-1} g_k$. A practical choice of B_k is discussed in section 4.1.

Relaxed Armijo Condition with an Error-Absorbing Term (line 10): The second component is the computation of the step size α_k using a relaxed Armijo condition, which is defined as follows:

$$\begin{cases} \bar{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k \geq \bar{f}(x_k + \alpha_k d_k), \\ \Delta_k = \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k)), \end{cases} \quad (3)$$

Algorithm 1: Noise Tolerant Regularized Quasi-Newton Method

Input: $x_0, 0 < k_{\max}, 0 \leq \varepsilon_f < 1, 0 < \theta_{\min} \leq \theta_{\max}$ and $0 < \varsigma < 1$

- 1 $k \leftarrow 0, g_0 \leftarrow g(x_0), K^0 \leftarrow \emptyset, K^+ \leftarrow \emptyset$
- 2 **for** $k = 0, 1, 2, \dots, k_{\max} - 1$ **do**
- 3 **if** $\min_{j \in K^0, j \leq k} (\bar{f}(x_j) - \Delta_j) \geq \bar{f}(x_k)$ **then**
- 4 $K^0 \leftarrow K^0 \cup \{k\}$
- 5 $\mu_k \leftarrow 0$
- 6 **else**
- 7 $K^+ \leftarrow K^+ \cup \{k\}$
- 8 $\mu_k \leftarrow \theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ (see section 4.3).
- 9 $d_k \leftarrow -(B_k + \mu_k I)^{-1} g_k$ with B_k (see section 4.1).
- 10 Find α_k and Δ_k satisfying eq. (3) by Algorithm 2.
- 11 $x_{k+1} \leftarrow x_k + \alpha_k d_k, g_{k+1} \leftarrow g(x_{k+1})$
- 12 **end**

Algorithm 2: Backtracking Line Search with Relaxed Armijo Condition

Input: $x_k \in \mathbb{R}^n, d_k \in \mathbb{R}^n, g_k \in \mathbb{R}^n, 0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$

- 1 $\alpha_k \leftarrow 1$
- 2 $\Delta_k \leftarrow \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k))$
- 3 **while** $\bar{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k < \bar{f}(x_k + \alpha_k d_k)$ **do**
- 4 $\alpha_k \leftarrow \beta \alpha_k$ with $\beta \in [\beta_{\min}, \beta_{\max}]$ (see section 4.2).
- 5 $\Delta_k \leftarrow \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k))$
- 6 **end**
- 7 **return** α_k, Δ_k

where Δ_k is the error-absorbing term that is recomputed for each α_k . This relaxation guarantees the existence of $\alpha_k \in (0, 1]$ even in the presence of errors in $\bar{f}$. When d_k is a descent direction ($g_k^\top d_k < 0$), eq. (3) ensures that a temporary increase of $\bar{f}$ is at most Δ_k . Since Δ_k depends on $-\bar{f}(x_k + \alpha_k d_k)$ but not on $\bar{f}(x_k)$, Δ_k does not increase when $\bar{f}(x_k + \alpha_k d_k)$ becomes large. This prevents unbounded growth of Δ_k .

Adaptive Update of the Regularization Parameter (lines 3–8): The third component is the adaptive update of the regularization parameter μ_k . A larger μ_k yields a more conservative step, while a smaller μ_k allows a more aggressive quasi-Newton step, especially when $\mu_k = 0$. In practical settings, $\mu_k = 0$ is preferred, since this allows us to fully exploit the quasi-Newton approximation and typically leads to practically faster convergence. Let us partition the iteration indices $K := \{0, 1, 2, \dots, k_{\max} - 1\}$ into two sets:

$$K^0 := \{k \in K \mid \mu_k = 0\}, \quad K^+ := \{k \in K \mid \mu_k > 0\}. \quad (4)$$

We set $\mu_k = 0$ if and only if the following condition is satisfied:

$$\min_{j \in K^0, j < k} (\bar{f}(x_j) - \Delta_j) \geq \bar{f}(x_k). \quad (5)$$

Note that $k = 0$ satisfies eq. (5) since K^0 is empty. This condition restricts $\mu_k = 0$ to iterations for which a sufficient decrease in $\bar{f}$ has been observed. **Thus, although $\bar{f}$ can temporarily increase, every iteration in K^0 must improve the previous error-adjusted reference value. When this test fails, the method instead enters K^+ and uses the OFFO-based regularization update below.** Otherwise, to ensure convergence to first-order stationary points, we apply an OFFO-based update

$$\mu_k = \theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}, \quad (6)$$

as described in section 1.2. Since OFFO schemes do not rely on function values, this update is robust to numerical errors in $\bar{f}$.

3 Theoretical Analysis

In this section, we establish the global convergence properties of Algorithm 1 under the problem setting in section 1.3, namely, eqs. (1) and (2).

3.1 Assumptions

In addition to the exact-gradient assumption stated in Assumption 1, we make the following three **structural** assumptions. The first assumption ensures that the objective function is bounded below.

Assumption 2 *The function f is bounded below, meaning that for all $x \in \mathbb{R}^n$,*

$$f(x) \geq f^* := \inf_{x \in \mathbb{R}^n} f(x) > -\infty.$$

By the triangle inequality and eq. (1), we have $\bar{f}(x) \geq f(x) - \varepsilon_f \max(1, |f(x)|)$. Since the function $g(t) = t - \varepsilon_f \max(1, |t|)$ is non-decreasing for $0 \leq \varepsilon_f < 1$, it follows that $\inf_x g(f(x)) = g(f^*)$. Consequently, we can also lower bound $\bar{f}$ as

$$\bar{f}(x) \geq \bar{f}^* := \inf_{x \in \mathbb{R}^n} (f(x) - \varepsilon_f \max(1, |f(x)|)) = f^* - \varepsilon_f \max(1, |f^*|) > -\infty. \quad (7)$$

The second assumption ensures smoothness of the objective function.

Assumption 3 *The function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is L -smooth, meaning that there exists a constant $L > 0$ such that for all $x, y \in \mathbb{R}^n$,*

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2.$$

The third assumption requires the matrices $\{B_k\}_{k=0}^{k_{\max}-1}$ to be uniformly positive definite and bounded.

Assumption 4 *There exist constants $0 < m \leq M$ such that all B_k satisfy*

$$mI \preceq B_k \preceq MI.$$

As discussed in section 4, the shifted raw L-BFGS construction with selected curvature pairs guarantees Assumption 4. By Assumption 4, we can derive useful bounds involving g_k and d_k . Using $d_k = -(B_k + \mu_k I)^{-1} g_k$ (line 9) and Assumption 4 asserting $(m + \mu_k)I \preceq B_k + \mu_k I \preceq (M + \mu_k)I$, we obtain the following bounds:

$$-g_k^\top d_k = d_k^\top (B_k + \mu_k I) d_k \geq (m + \mu_k) \|d_k\|^2 \geq m \|d_k\|^2, \quad (8)$$

$$-g_k^\top d_k = g_k^\top (B_k + \mu_k I)^{-1} g_k \geq \frac{1}{M + \mu_k} \|g_k\|^2, \quad (9)$$

$$\|d_k\|^2 = \|(B_k + \mu_k I)^{-1} g_k\|^2 \leq \frac{1}{(m + \mu_k)^2} \|g_k\|^2. \quad (10)$$

3.2 Error Bound on Function Evaluations

We first establish useful properties of the error-absorbing term Δ_k defined in eq. (3). In the following, we define the actual and observed function decreases as

$$\delta_k := f(x_k) - f(x_{k+1}), \quad (11)$$

$$\bar{\delta}_k := \bar{f}(x_k) - \bar{f}(x_{k+1}). \quad (12)$$

The first property states that the **difference** between the actual **decrease** δ_k and the observed **decrease** $\bar{\delta}_k$ can be bounded by Δ_k .

Lemma 1 *If x_k and x_{k+1} satisfy $f(x_{k+1}) \leq f(x_k)$, it holds that*

$$\delta_k \leq \bar{\delta}_k + \Delta_k.$$

Proof We start from general observations. We first prove that, for all $x \in \mathbb{R}^n$,

$$\begin{cases} \max(1, +f(x)) \leq \frac{1}{1-\varepsilon_f} \max(1, +\bar{f}(x)), \\ \max(1, -f(x)) \leq \frac{1}{1-\varepsilon_f} \max(1, -\bar{f}(x)). \end{cases} \quad (13)$$

We prove the inequality with the plus sign. If $+f(x) \leq 1$, then $\max(1, +f(x)) = 1 \leq \frac{1}{1-\varepsilon_f} \leq \frac{1}{1-\varepsilon_f} \max(1, +\bar{f}(x))$. If $+f(x) > 1$, eq. (1) implies

$$|f(x) - \bar{f}(x)| \leq \varepsilon_f \max(1, |f(x)|) = +\varepsilon_f f(x) \quad (14)$$

and thus $f(x) - \bar{f}(x) \leq +\varepsilon_f f(x)$. Hence,

$$\max(1, +f(x)) = +f(x) \leq +\frac{1}{1-\varepsilon_f} \bar{f}(x) \leq \frac{1}{1-\varepsilon_f} \max(1, +\bar{f}(x)).$$

The case with the negative sign can be shown analogously, using $-f(x) + \bar{f}(x) \leq -\varepsilon_f f(x)$, which follows from eq. (14). This completes the proof of eq. (13).

We also have the following general bound for all $k \in K$:

$$\begin{aligned}
|\delta_k - \bar{\delta}_k| &= |(f(x_k) - \bar{f}(x_k)) - (f(x_{k+1}) - \bar{f}(x_{k+1}))| && \text{(rearrangement)} \\
&\leq |f(x_k) - \bar{f}(x_k)| + |f(x_{k+1}) - \bar{f}(x_{k+1})| && \text{(triangle inequality)} \\
&\leq \varepsilon_f \max(1, |f(x_k)|) + \varepsilon_f \max(1, |f(x_{k+1})|) && \text{(eq. (1))} \\
&\leq 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|). && \text{(max property)} \quad (15)
\end{aligned}$$

With these preparations, we now prove the lemma. From the condition $f(x_{k+1}) \leq f(x_k)$, eq. (15) can be further bounded as

$$\begin{aligned}
2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) &= 2\varepsilon_f \max(1, f(x_k), -f(x_{k+1})) && (f(x_{k+1}) \leq f(x_k)) \\
&\leq \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) && \text{(eq. (13))} \\
&= \Delta_k, && \text{(eq. (3))}
\end{aligned}$$

which yields the desired inequality with $\delta_k - \bar{\delta}_k \leq |\delta_k - \bar{\delta}_k| \leq \Delta_k$. $\square$

We also derive the following lemma for later use.

Lemma 2 *Under Assumption 4, it holds that*

$$\bar{\delta}_k \leq \delta_k + \frac{1 + \varepsilon_f}{1 - \varepsilon_f} \Delta_k.$$

Proof Since $-g_k^\top d_k \geq 0$ from Assumption 4 and eq. (8), the relaxed Armijo condition (eq. (3)) implies

$$\bar{f}(x_k) - \bar{f}(x_{k+1}) \geq -c\alpha_k g_k^\top d_k - \Delta_k \geq -\Delta_k. \quad (16)$$

Thus, using the general results in eq. (15) and $\Delta_k \geq 0$, we can further bound as

$$\begin{aligned}
|\delta_k - \bar{\delta}_k| &\leq 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) && \text{(eq. (15))} \\
&\leq \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k), \bar{f}(x_{k+1}), -\bar{f}(x_{k+1})) && \text{(eq. (13))} \\
&\leq \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k) + \Delta_k, -\bar{f}(x_{k+1}) + \Delta_k) && \text{(eq. (16))} \\
&\leq \frac{2\varepsilon_f}{1 - \varepsilon_f} (\max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) + \Delta_k) \\
&= \left(1 + \frac{2\varepsilon_f}{1 - \varepsilon_f}\right) \Delta_k = \frac{1 + \varepsilon_f}{1 - \varepsilon_f} \Delta_k, && \text{(eq. (3))}
\end{aligned}$$

which yields the desired inequality with $\bar{\delta}_k - \delta_k \leq |\delta_k - \bar{\delta}_k| \leq \frac{1 + \varepsilon_f}{1 - \varepsilon_f} \Delta_k$. $\square$

3.3 Backtracking Stage

We next analyze the line search in Algorithm 2. Recall that the parameter c in Algorithm 2 satisfies $0 < c < 1$ and $0 < m$ from Assumption 4.

Lemma 3 *Under Assumptions 3 and 4, the backtracking procedure in Algorithm 2 terminates whenever the step size α_k satisfies*

$$\alpha_k \leq \frac{2(1-c)m}{L}. \quad (17)$$

Proof Under the L -smoothness of f (Assumption 3), we always have

$$\begin{aligned} f(x_k) - f(x_k + \alpha_k d_k) &\geq -\alpha_k g_k^\top d_k - \frac{L}{2} \alpha_k^2 \|d_k\|^2 \\ &= -c \alpha_k g_k^\top d_k + \alpha_k \left(-(1-c) g_k^\top d_k - \frac{L \alpha_k}{2} \|d_k\|^2 \right). \end{aligned} \quad (18)$$

From Assumption 4 and eq. (8), we have $-g_k^\top d_k \geq m \|d_k\|^2$. Under eq. (17), we have

$$-(1-c) g_k^\top d_k - \frac{L \alpha_k}{2} \|d_k\|^2 \geq \left((1-c)m - \frac{L \alpha_k}{2} \right) \|d_k\|^2 \geq 0. \quad (19)$$

Combining eqs. (18) and (19) yields

$$f(x_k) - f(x_k + \alpha_k d_k) \geq -c \alpha_k g_k^\top d_k.$$

Since $-g_k^\top d_k \geq 0$ from Assumption 4 and eq. (8), the condition of Lemma 1 is satisfied with $x_{k+1} = x_k + \alpha_k d_k$. Thus, we obtain $\delta_k \leq \bar{\delta}_k + \Delta_k$, i.e.,

$$-c \alpha_k g_k^\top d_k \leq f(x_k) - f(x_k + \alpha_k d_k) \leq \bar{f}(x_k) - \bar{f}(x_k + \alpha_k d_k) + \Delta_k.$$

This inequality means that the relaxed Armijo condition (eq. (3)) is satisfied and thus the backtracking procedure terminates. This completes the proof. $\square$

Now, let us analyze Algorithm 2 using Lemma 3. Define

$$\bar{\alpha} := \beta_{\min} \min \left(\frac{2(1-c)m}{L}, 1 \right) > 0.$$

The following states the key property of the backtracking procedure.

Proposition 1 *Under Assumptions 3 and 4, the backtracking line search in Algorithm 2 terminates after finitely many rejections, and the step size α_k satisfies*

$$\alpha_k \geq \bar{\alpha}.$$

Proof Line 4 of Algorithm 2 multiplies α_k by β at each rejection, where $0 < \beta_{\min} \leq \beta \leq \beta_{\max} < 1$. For the number of backtracking rejections r , we have $\alpha_k \leq \beta_{\max}^r$. Thus, r is finite since the backtracking procedure terminates for sufficiently small α_k by Lemma 3. If $r = 0$, $\alpha_k = 1 \geq \bar{\alpha}$. Otherwise, $\alpha_k \geq \beta_{\min} \alpha'_k$, where α'_k is the last rejected step size. By Lemma 3, we have $\alpha'_k > \frac{2(1-c)m}{L}$, and thus

$$\alpha_k \geq \beta_{\min} \alpha'_k > \beta_{\min} \frac{2(1-c)m}{L} \geq \bar{\alpha}.$$

This completes the proof. $\square$

3.4 Bound for the case $\mu_k = 0$

In this subsection, we lower bound the sum of actual function decreases δ_k over $k \in K^0$. Recall that K^0 denotes the set of iteration indices k such that $\mu_k = 0$, as in eq. (4). To this end, we establish that the sum of Δ_k over $k \in K^0$ is upper bounded, where Δ_k is the error-absorbing term in the relaxed Armijo condition (eq. (3)). We introduce the following constant $\Delta_{\max}$, which serves as an upper bound for Δ_k of $k \in K^0$.

$$\Delta_{\max} := \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_0), -\bar{f}^*).$$

Lemma 4 *Under Assumptions 2 to 4, the sum of Δ_k over $k \in K^0$ satisfies*

$$\sum_{k \in K^0} \Delta_k \leq \bar{f}(x_0) - \bar{f}^* + \Delta_{\max}.$$

Proof Let $(k_i)_{i=0}^\ell$ with $\ell \geq 0$ be the elements of K^0 in increasing order. We have $\bar{f}(x_{k_\ell}) \leq \bar{f}(x_0)$ from eq. (5) with $k = k_\ell$, where the case $\ell = 0$ follows from $k_0 = 0$. Combining this with Assumption 2, we have $\Delta_{k_\ell} \leq \Delta_{\max}$. For all $i < \ell$, eq. (5) with $j = k_i$ and $k = k_{i+1}$ yields

$$\Delta_{k_i} \leq \bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+1}}), \quad (20)$$

and summing Δ_k over $k \in K^0$ gives

$$\begin{aligned} \sum_{k \in K^0} \Delta_k &\leq \sum_{i=0}^{\ell-1} (\bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+1}})) + \Delta_{k_\ell} && \text{(eq. (20))} \\ &= \bar{f}(x_0) - \bar{f}(x_{k_\ell}) + \Delta_{k_\ell} && \text{(telescoping sum)} \\ &\leq \bar{f}(x_0) - \bar{f}^* + \Delta_{\max}. && (-\bar{f}(x_{k_\ell}) \leq -\bar{f}^* \text{ and } \Delta_{k_\ell} \leq \Delta_{\max}) \end{aligned}$$

This completes the proof. $\square$

We now bound the sum of $\delta_k = f(x_k) - f(x_{k+1})$ over $k \in K^0$ with $\|g_k\|^2$ as follows.

Lemma 5 *Under Assumptions 2 to 4, the sum of δ_k over $k \in K^0$ satisfies*

$$\sum_{k \in K^0} \delta_k \geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 - \frac{2}{1 - \varepsilon_f} (\bar{f}(x_0) - \bar{f}^* + \Delta_{\max}).$$

Proof The relaxed Armijo condition (eq. (3)) at iteration $k \in K^0$ yields

$$\begin{aligned} \bar{\delta}_k + \Delta_k &= \bar{f}(x_k) - \bar{f}(x_{k+1}) + \Delta_k && \text{(eq. (12))} \\ &\geq -c\alpha_k g_k^\top d_k && \text{(eq. (3))} \\ &\geq \frac{c\bar{\alpha}}{M + \mu_k} \|g_k\|^2 && \text{(Proposition 1 and eq. (9))} \\ &= \frac{c\bar{\alpha}}{M} \|g_k\|^2. && (\mu_k = 0 \text{ since } k \in K^0) \end{aligned} \quad (21)$$

Using Lemma 2, we also have

$$\bar{\delta}_k + \Delta_k \leq \delta_k + \frac{1 + \varepsilon_f}{1 - \varepsilon_f} \Delta_k + \Delta_k = \delta_k + \frac{2}{1 - \varepsilon_f} \Delta_k. \quad (22)$$

Then, combining eqs. (21) and (22) and summing over $k \in K^0$ gives

$$\sum_{k \in K^0} \frac{c\bar{\alpha}}{M} \|g_k\|^2 \leq \sum_{k \in K^0} \delta_k + \frac{2}{1 - \varepsilon_f} \sum_{k \in K^0} \Delta_k \leq \sum_{k \in K^0} \delta_k + \frac{2}{1 - \varepsilon_f} (\bar{f}(x_0) - \bar{f}^* + \Delta_{\max}),$$

where the last inequality follows from Lemma 4. Rearranging this completes the proof. $\square$

3.5 Bound for the case $\mu_k > 0$

Next, we show that the sum of δ_k over $k \in K^+$ is lower bounded. Recall that K^+ is the set of iteration indices k such that $\mu_k > 0$ as defined in eq. (4), and that μ_k is defined as $\theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ in line 8 of Algorithm 1.

Before proceeding to the main proof, we present Lemma 6, which collects standard inequalities from the AdaGrad-Norm analysis [42, Lemma 3.2] [19, Lemma 3.1]. Its proof is deferred to ??.

Lemma 6 *In Algorithm 1, we have*

$$\begin{aligned} \sum_{k \in K^+} \frac{\|g_k\|^2}{\mu_k^2} &\leq \frac{1}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right), \\ \sum_{k \in K^+} \frac{\|g_k\|^2}{\mu_k} &\geq \frac{1}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right). \end{aligned}$$

We now lower bound the sum of δ_k over $k \in K^+$ with $\|g_k\|^2$. We define the following constants for later use:

$$C_1 := \frac{\bar{\alpha}}{\theta_{\max}}, \quad C_2 := \frac{M\bar{\alpha} + L/2}{\theta_{\min}^2}.$$

Lemma 7 *Under Assumptions 3 and 4, the sum of δ_k over $k \in K^+$ satisfies*

$$\sum_{k \in K^+} \delta_k \geq C_1 \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right) - C_2 \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right).$$

Proof From the L -smoothness (Assumption 3), for $k \in K^+$ we have

$$\begin{aligned} \delta_k &= f(x_k) - f(x_k + \alpha_k d_k) && \text{(eq. (11) and } x_{k+1} = x_k + \alpha_k d_k) \\ &\geq -\alpha_k g_k^\top d_k - \frac{\alpha_k^2 L}{2} \|d_k\|^2 && \text{(Assumption 3)} \end{aligned}$$

$$\begin{aligned}
&\geq \left(\frac{\alpha_k}{M + \mu_k} - \frac{\alpha_k^2 L}{2(m + \mu_k)^2} \right) \|g_k\|^2 \quad (\text{eqs. (9) and (10)}) \\
&\geq \left(\frac{\bar{\alpha}}{M + \mu_k} - \frac{L}{2(m + \mu_k)^2} \right) \|g_k\|^2. \quad (\text{Proposition 1 and } \alpha_k \leq 1) \quad (23)
\end{aligned}$$

We can further bound the terms in eq. (23) as

$$\frac{\bar{\alpha}}{M + \mu_k} = \frac{\bar{\alpha}}{\mu_k} \frac{1}{1 + \frac{M}{\mu_k}} \geq \frac{\bar{\alpha}}{\mu_k} \left(1 - \frac{M}{\mu_k} \right) = \frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha}}{\mu_k^2}, \quad \frac{L}{2(m + \mu_k)^2} \leq \frac{L}{2\mu_k^2}. \quad (24)$$

Thus, applying eq. (24) to eq. (23) yields

$$\delta_k \geq \left(\frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha} + L/2}{\mu_k^2} \right) \|g_k\|^2. \quad (25)$$

Summing eq. (25) over $k \in K^+$ and applying Lemma 6 yield

$$\begin{aligned}
\sum_{k \in K^+} \delta_k &\geq \sum_{k \in K^+} \left(\frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha} + L/2}{\mu_k^2} \right) \|g_k\|^2 \\
&\geq \frac{\bar{\alpha}}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right) - \frac{M\bar{\alpha} + L/2}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right),
\end{aligned}$$

which yields the desired result. $\square$

3.6 Global Convergence Rate

Finally, we combine the results for $k \in K^0$ stated in Lemma 5 and $k \in K^+$ stated in Lemma 7 to derive an upper bound on the total sum of $\|g_k\|^2$. We define a constant F as follows:

$$F := f(x_0) - f^* + \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right) + C_1 \sqrt{\varsigma} - C_2 \ln(\varsigma). \quad (26)$$

Then, we can obtain the following unified bound **for the two types of iterations**.

Lemma 8 *Under Assumptions 2 to 4, the sum of $\|g_k\|^2$ over $k \in K = K^0 \cup K^+$ satisfies*

$$F \geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + C_1 \sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - C_2 \ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right),$$

Proof Since $f(x_{k_{\max}}) \geq f^*$ by Assumption 2 and $K = K^0 \cup K^+$, we have

$$f(x_0) - f^* \geq f(x_0) - f(x_{k_{\max}}) = \sum_{k \in K} \delta_k = \sum_{k \in K^0} \delta_k + \sum_{k \in K^+} \delta_k.$$

From Lemmas 5 and 7, we can further bound as

$$f(x_0) - f^* \geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 - \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right)$$

$$+ C_1 \sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - C_1 \sqrt{\varsigma} - C_2 \ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) + C_2 \ln(\varsigma).$$

Substituting eq. (26) yields the desired bound, concluding the proof. $\square$

Now, we establish the following **convergence-rate and iteration-complexity result** for Algorithm 1.

Theorem 1 *Under Assumptions 1 to 4, the iterates generated by Algorithm 1 satisfy*

$$\frac{1}{k_{\max}} \sum_{k=0}^{k_{\max}-1} \|g_k\|^2 = \mathcal{O} \left(\frac{1}{k_{\max}} \right).$$

Consequently, for every $0 < \varepsilon \leq 1$, the algorithm generates an iterate satisfying $\|g_k\| \leq \varepsilon$ within $\mathcal{O}(1/\varepsilon^2)$ iterations.

Proof Let us define the function $\phi: \mathbb{R}_{>0} \rightarrow \mathbb{R}$ as

$$\phi(G) := \frac{C_1}{2} \sqrt{G} - C_2 \ln(G). \quad (27)$$

Since $\phi'(G) = \frac{C_1}{4\sqrt{G}} - \frac{C_2}{G}$, its minimum over $G > 0$ is attained at $G^* := \left(\frac{4C_2}{C_1} \right)^2$ with

$$\phi(G^*) = 2C_2 \left(1 - \ln \left(\frac{4C_2}{C_1} \right) \right) = 2C_2 \ln \left(\frac{eC_1}{4C_2} \right),$$

where e is the base of the natural logarithm. Thus, Lemma 8 yields

$$\begin{aligned} F &\geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} + \phi \left(\sum_{k \in K^+} \|g_k\|^2 + \varsigma \right) \\ &\geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} + \phi(G^*), \end{aligned}$$

which is equivalent to

$$0 \leq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} \leq F' \quad (28)$$

where

$$\begin{aligned} F' &:= F - \phi(G^*) \\ &= f(x_0) - f^* + \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right) + C_1 \sqrt{\varsigma} - C_2 \ln(\varsigma) - 2C_2 \ln \left(\frac{eC_1}{4C_2} \right). \end{aligned}$$

It remains to bound $\sum_{k \in K} \|g_k\|^2 = \sum_{k \in K^0} \|g_k\|^2 + \sum_{k \in K^+} \|g_k\|^2$ under eq. (28). More generally, let $a, b > 0$, $c \geq 0$, and $x, y \geq 0$, and consider maximizing $x + y$ subject to $ax + b\sqrt{y} \leq c$. The maximum is attained on the boundary $ax + b\sqrt{y} = c$ because the objective is increasing in both variables. On this boundary, $y = (c - ax)^2/b^2$, so the objective is convex over $0 \leq x \leq c/a$ and attains its maximum at an endpoint.

Evaluating the two endpoints gives $x + y \leq \max\{c/a, c^2/b^2\}$. Applying this bound to eq. (28) yields

$$\frac{1}{k_{\max}} \sum_{k=0}^{k_{\max}-1} \|g_k\|^2 \leq \frac{1}{k_{\max}} \max \left\{ \frac{MF'}{c\bar{\alpha}}, \frac{4(F')^2}{C_1^2} \right\}. \quad (29)$$

Since the minimum is no larger than the average, eq. (29) also guarantees an iterate satisfying $\|g_k\| \leq \varepsilon$ whenever

$$k_{\max} \geq \frac{1}{\varepsilon^2} \max \left\{ \frac{MF'}{c\bar{\alpha}}, \frac{4(F')^2}{C_1^2} \right\}. \quad (30)$$

This completes the proof. $\square$

As a remark, the function $\phi(G)$ in eq. (27) is closely related to the Lambert W function [8, 19]. For $x \in [-1/e, 0)$, the equation $ye^y = x$ admits two real negative solutions, and the smaller one is given by the second branch of the Lambert W function, $y = W_{-1}(x) < 0$. Using this function, the equation $\phi(G) = 0$ can be solved in closed form, which leads to a sharper bound in Theorem 1. Since this is not essential for the main argument, we omit the details and refer to [19].

We next interpret eq. (29) in the proof of Theorem 1. The contribution of the indices in K^0 is analogous to the classical analysis of gradient descent. Indeed, for gradient descent with fixed step size $\alpha = 1/L$, one has

$$\frac{1}{k_{\max}} \sum_{k=0}^{k_{\max}-1} \|g_k\|^2 \leq \frac{1}{k_{\max}} \frac{2(f(x_0) - f^*)}{\alpha}.$$

Thus, the term associated with K^0 reflects the standard $\mathcal{O}(f(x_0) - f^*)$ dependence of first-order methods since F' contains the initial gaps $f(x_0) - f^*$ and $\bar{f}(x_0) - \bar{f}^*$. In contrast, the contribution of K^+ resembles the behavior of AdaGrad-Norm, whose convergence bound scales quadratically with the initial optimality gap [42].

Finally, Theorem 1 implies the **iteration-complexity bound**

$$\min \{k \mid \|\nabla f(x_k)\| \leq \varepsilon_{\text{gtol}}\} = \mathcal{O}(1/\varepsilon_{\text{gtol}}^2).$$

The order $\mathcal{O}(1/\varepsilon_{\text{gtol}}^2)$ is worst-case optimal for first-order stationarity on the class of L -smooth nonconvex functions even when function values are exact [5]. Hence, bounded nonvanishing errors in the function values used by our hybrid mechanism do not worsen the best possible dependence on $\varepsilon_{\text{gtol}}$, although they affect the constants below. More explicitly, eq. (30) gives the sufficient bound

$$k_{\max} \geq \frac{1}{\varepsilon_{\text{gtol}}^2} \max \left\{ \frac{MF'}{c\bar{\alpha}}, \frac{4(F')^2}{C_1^2} \right\}.$$

Here $\bar{\alpha}$ depends only on c, L, m and the backtracking bounds, $C_1 = \bar{\alpha}/\theta_{\max}$, $C_2 = (M\bar{\alpha} + L/2)/\theta_{\min}^2$, and F' is displayed in the proof and contains ε_f , ζ , the initial true and observed optimality gaps, and $\Delta_{\max}$. The construction in section 4.1 gives $m = \omega_{\min}$ and $M = (1 + (1 + p)\Lambda)\rho_0$, so the memory length and fixed initial scale enter

the sufficient iteration bound through M . Thus, the hidden constant in the $\mathcal{O}\left(1/\varepsilon_{\text{gtol}}^2\right)$ bound is independent of $\varepsilon_{\text{gtol}}$. The guarantee deteriorates as the permitted function error approaches 1 through the factor $(1 - \varepsilon_f)^{-1}$.

4 Practical Choices of Algorithmic Variables

In sections 2 and 3, we introduced Algorithm 1 and established its convergence properties. In this section, we discuss practical choices of algorithmic variables and parameters that lead to fast and stable performance. The quantities to be discussed here are the matrices B_k , the step sizes α_k , and the regularization parameters μ_k .

4.1 Approximate Hessian B_k

We first discuss the construction of the matrices B_k used in line 9 of Algorithm 1. The purpose of this subsection is to show that a shifted raw L-BFGS construction with modified and carefully selected curvature pairs can satisfy Assumption 4.

Construction of B_k : We recall the L-BFGS construction [4, 26, 31]. For an iteration k , we define

$$s_k := x_{k+1} - x_k, \quad y_k := g_{k+1} - g_k$$

where g_k and g_{k+1} are the gradients at x_k and x_{k+1} , respectively. Starting from an initial matrix, an approximate Hessian is obtained by successively applying BFGS updates to a sequence of curvature pairs. Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a collection of curvature pairs ordered so that $k_0 < \dots < k_{p-1}$. Starting from the initial matrix $B^{(0)} = \gamma I$ with $\gamma = \left\| y_{k_{p-1}} \right\|^2 / y_{k_{p-1}}^\top s_{k_{p-1}}$, we define $\text{BFGS}(\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1})$ as the matrix $B^{(p)}$ obtained by sequentially applying the BFGS update as

$$B^{(j+1)} = B^{(j)} - \frac{B^{(j)} s_{k_j} s_{k_j}^\top B^{(j)}}{s_{k_j}^\top B^{(j)} s_{k_j}} + \frac{y_{k_j} y_{k_j}^\top}{y_{k_j}^\top s_{k_j}}$$

for $j = 0, 1, \dots, p-1$, where each curvature pair (s_{k_j}, y_{k_j}) is applied in order. Here, $p > 0$ denotes the maximum number of stored curvature pairs and is a fixed small integer in the L-BFGS method. When fewer than p curvature pairs are available, the BFGS update is applied using all currently available pairs.

We now give a sufficient condition for the upper spectral bound in Assumption 4, based on variants of existing results [13, Theorem 5.5] [15, Lemma 1]. Its proof is deferred to ??.

Proposition 2 *Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be curvature pairs with $s_{k_i} \neq 0$. Suppose that there exists a constant $\Lambda > 0$ such that every pair (s, y) in this collection satisfies*

$$y^\top s \geq \frac{1}{\Lambda} \|y\|^2 > 0. \quad (31)$$

Then

$$0 \prec \text{BFGS}(\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}) \preceq (1 + p)\Lambda I. \quad (32)$$

Proposition 2 supplies the upper bound, while a fixed diagonal shift will supply the lower bound in Assumption 4. We first apply a scalar Powell-type damping to a candidate pair with $y_{k_i}^\top s_{k_i} < 0$. For a parameter $\eta_{\text{damp}} \in (0, 1)$, let $b_i = \|y_{k_i}\|^2 / |y_{k_i}^\top s_{k_i}|$ and set

$$\bar{y}_{k_i} = \theta_i^{\text{damp}} y_{k_i} + (1 - \theta_i^{\text{damp}}) b_i s_{k_i}, \quad \theta_i^{\text{damp}} = \frac{\eta_{\text{damp}} b_i \|s_{k_i}\|^2}{b_i \|s_{k_i}\|^2 - y_{k_i}^\top s_{k_i}} \in [0, 1).$$

If $y_{k_i}^\top s_{k_i} \geq 0$, we set $\bar{y}_{k_i} = y_{k_i}$. Candidates for which the damping is numerically unsafe are discarded.

We then select the damped pairs using a cautious update in the spirit of [24, 27, 28]. Define the fixed initial scale and diagonal shift by

$$\rho_0 := \max\{1, \|g_0\|\}, \quad \omega_{\min} := \min\{1, \sqrt{\varsigma}\} > 0.$$

For a fixed $\Lambda > 0$, a candidate pair $(s, \bar{y})$ is retained only if

$$\bar{y}^\top s \geq \frac{\|\bar{y}\|^2}{\Lambda \rho_0} > 0. \quad (33)$$

Consequently, every retained pair $(s, \bar{y}/\rho_0)$ satisfies eq. (31), and the homogeneity of the BFGS update gives

$$\text{BFGS}(\{(s_{k_i}, \bar{y}_{k_i})\}) = \rho_0 \text{BFGS}(\{(s_{k_i}, \bar{y}_{k_i}/\rho_0)\}).$$

Using at most p retained pairs, we define

$$B_k = \begin{cases} \omega_{\min} I + \text{BFGS}(\{(s_{k_i}, \bar{y}_{k_i})\}), & \text{if the memory is nonempty,} \\ (\omega_{\min} + \rho_0) I, & \text{otherwise.} \end{cases} \quad (34)$$

It follows from Proposition 2, the homogeneity identity, and $\omega_{\min} \leq 1 \leq \rho_0$ that

$$\omega_{\min} I \preceq B_k \preceq (1 + (1 + p)\Lambda) \rho_0 I.$$

Thus, Assumption 4 holds with the iteration-independent constants $m = \omega_{\min}$ and $M = (1 + (1 + p)\Lambda) \rho_0$. This argument uses neither convexity nor cocoercivity of ∇f . In our implementation, we use $p = 10$, $\Lambda = 10^8$, and $\eta_{\text{damp}} = 0.8$, and strengthen eq. (33) with the absolute safeguard $\bar{y}^\top s \geq 10^{-16}$ to reject numerically degenerate pairs.

Exact Compact Solve: We use a compact representation of the raw L-BFGS matrix to solve the regularized system exactly [23]. Specifically, we compute $d_k = -(B_k + \mu_k I)^{-1} g_k$ for the matrix in eq. (34), without absorbing the additive regularization into the curvature pairs. This exact compact solve therefore coincides with the direction analyzed in section 3. When the memory is empty, the same definition reduces to $d_k = -g_k/(\rho_0 + \omega_{\min} + \mu_k)$. If the compact factorization fails numerically or does not produce a finite descent direction, the implementation uses this zero-memory direction, which also corresponds to a matrix satisfying Assumption 4.

Shifted-Pair Approximation: For comparison, the implementation retains the simpler shifted-pair approximation

$$B_k + \mu_k I \approx \text{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + (\mu_k + \omega_{\min})s_{k_i}\}_{i=0}^{p-1}). \quad (35)$$

The variants using this approximation are denoted by the suffix -SP and are treated as heuristic variants rather than as the implementations covered directly by the analysis. They retain the standard two-loop recursion and are included as lower-overhead practical alternatives to the exact compact solve.

Function-Based Modified Secant Variants: Separately, there exist function-based modified secant conditions [1, 20, 44, 47] that adjust the secant equation using available function values when those quantities are reliable. Such adjustments aim to improve practical convergence behavior. We partially adopt these function-based modifications following prior work [45, 48]. The suffix -MS in section 5 indicates the use of this technique. Because these modifications are well established, we omit low-level implementation details here. For the proposed -MS variants, a scale-aware safeguard suppresses the function-based correction whenever it would dominate the raw gradient difference. Further specifics are available in the cited references and in our open-source implementation.

4.2 Step Size α_k

We next discuss the adjustment of the step size α_k in line 4 of Algorithm 2. As indicated in the pseudo-code, we choose α_k using interpolation of the objective function. Moré–Thuente line searches [29] are widely used in quasi-Newton methods [40], and they typically employ cubic or quadratic interpolation to select trial step sizes. In our implementation, we use an essentially identical procedure, except that the trial step size is obtained by clipping it to the interval $[\beta_{\min}\alpha_k, \beta_{\max}\alpha_k]$. For all numerical experiments, we set the Armijo parameter to $c = 10^{-4}$ and choose the interpolation bounds as $\beta_{\min} = 1/16$ and $\beta_{\max} = 15/16$.

Furthermore, when $\mu_k > 0$ and $\alpha_k = 1$, we introduce a one-time adjustment of the trial step size using gradient information. Let d_k be the search direction and g_k and g_{try} the gradients at the current and trial points. If the directional derivative along d_k changes sign, $d_k^\top g_k < 0$ and $d_k^\top g_{\text{try}} > 0$, and the trial gradient is sufficiently aligned with d_k , namely $d_k^\top g_{\text{try}} > 0.5\|d_k\|\|g_{\text{try}}\|$, we rescale α_k once by a secant-like factor,

$$\alpha_k \leftarrow \text{clip}\left(\alpha_k \frac{-d_k^\top g_k}{d_k^\top g_{\text{try}} - d_k^\top g_k}, \beta_{\min}\alpha_k, \beta_{\max}\alpha_k\right),$$

where $\text{clip}(x, a, b) := \min(\max(x, a), b)$ is a clipping function. When objective values are unreliable and backtracking and interpolation do not occur, this correction effectively refines the step size. After the adjustment, the relaxed Armijo condition is evaluated again, and ordinary backtracking continues if necessary. The clipping interval gives $\alpha_k \geq \beta_{\min}$ whenever this adjustment is applied, so the conservative lower bound $\bar{\alpha}$ in Proposition 1 and the subsequent convergence analysis remain valid.

4.3 Regularization Parameter μ_k

We finally discuss the adjustment of the regularization parameter μ_k in line 8 of Algorithm 1.

Regularization Update: The OFFO accumulator contains exactly the gradient norms generated during regularized iterations:

$$G_k^2 = \varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2.$$

For a parameter $\eta_\mu > 0$, we set

$$\mu_k = \text{clip}(\eta_\mu \|g_k\|, \theta_{\min} G_k, \theta_{\max} G_k).$$

This clipping rule gives $\mu_k = \theta_k G_k$ for some $\theta_k \in [\theta_{\min}, \theta_{\max}]$, as required by eq. (6). In our implementation, $\eta_\mu = 1/10$, $\theta_{\min} = 1/100$, $\theta_{\max} = 1$, and $\varsigma = 10^{-20}$, so the initial accumulator scale is $G_{-1} = \sqrt{\varsigma} = 10^{-10}$.

Optional Accumulator Restart: The restart was introduced as a practical heuristic rather than as part of the analyzed algorithm. The heuristic is motivated by the observation that continued growth of G_k can over-regularize later steps after the method has again obtained clear progress in the objective value. Resetting the accumulator after such progress is intended to reduce the effective regularization and recover faster quasi-Newton behavior. For a fixed threshold $\tau > 0$, the heuristic resets only the accumulator when the method returns from K^+ to K^0 and

$$\min_{j \in K^0, j < k} (\bar{f}(x_j) - \Delta_j) - \bar{f}(x_k) \geq \tau. \quad (36)$$

The sets K^0 and K^+ and the reference values in eq. (36) are not reset. The numerical experiments distinguish the no-restart and restarted variants. The former coincides with the analyzed accumulator, whereas the latter uses $\tau = 1$ and at most 10 restarts.

5 Numerical Experiments

We conducted numerical experiments to evaluate the practical performance of our proposed method and to compare it with existing optimization algorithms.

5.1 Methods

We compared the following methods in our experiments.

- (Ours): Our proposed regularized quasi-Newton method
 - We used Algorithms 1 and 2 with the details in section 4.
 - (Ours-MS) means the variant incorporating the modified secant condition described in section 4.1.

- (Ours-SP) means the heuristic variant replacing the exact compact solve with the shifted-pair approximation in eq. (35).
- (Ours-R) means the heuristic variant incorporating the optional accumulator restart described in section 4.3.
- (Line): Line search L-BFGS method
 - Our Python implementation ported from LBFGSpp [32].
 - The step size was determined using the Moré–Thuente line search, closely resembling SciPy’s implementation.
- (Reg): Regularized L-BFGS method [23]
 - A quadratic regularization-based quasi-Newton method that does not rely on line search and is conceptually closer to trust-region methods.
 - We wrapped the function `regLBFGS` with `solveNonmonotone` from [23].
 - (Reg-Sec) denotes the variant using the approximate computation in eq. (35). It wraps the `regLBFGSsec` function from [23].
- (SciPy): SciPy’s L-BFGS-B method [40]
 - An established L-BFGS method implemented in C and called from Python.
 - We used the default parameters, with the exception that `ftol` was set to 0 to avoid premature termination.
- (NTQN): Noise-tolerant quasi-Newton method [36, 43]
 - A method designed to accommodate inexact gradient information, as mentioned in section 1.3.
 - We used the common gradient-tolerance stopping rule rather than NTQN’s recommended termination mode, which yielded a substantially lower attainment rate under the common tolerance.
- (ASTR1-Adagrad): The purely first-order adagrad variant of ASTR1.
 - The parameter setting is the same as reported by Gratton et al. [19, Table 1].

In the following, each method is referred to by the name given in parentheses. Except for the first-order ASTR1-Adagrad baseline, all these methods are L-BFGS-based, and we set the number of stored curvature pairs to 10, the default value in SciPy’s L-BFGS-B implementation. All experiments were conducted on an Apple Mac Pro (Late 2013) with a 2.7 GHz 12-core Intel Xeon E5-2697 v2 processor and 64 GB of memory. All computations were performed using the CPU only.

5.2 Experimental Results

5.2.1 Test Problems and Settings

We evaluated all algorithms on unconstrained problems from the CUTEst benchmark collection [14], implemented in Python using the PyCUTEst interface [12]. All problems were initialized with the default starting points provided by the library. Following previous work [23], we excluded problems whose initial points were already stationary or whose function values or gradients contained NaN or INF. The retained problems have dimensions ranging from 2 to 123,200.

We organize the CUTEst experiments into the following two groups according to where their results are reported:

- *Artificial-noise experiments* on 220 problems using 64-bit arithmetic (results in section 5.2.3)
 - *Function-value noise*: We add independent noise $\xi_f \sim \text{unif}(-10^{-3}, 10^{-3})$ to each function evaluation while retaining exact gradients. We set the nominal error allowance to $\varepsilon_f = 10^{-2}$.
 - *Joint function-and-gradient noise*: We use the same function perturbation and additionally perturb each gradient component independently by $\xi_g^{(i)} \sim \text{unif}(-10^{-3}, 10^{-3})$. We again set $\varepsilon_f = 10^{-2}$.
 - *Error-bound sensitivity*: Using the function-value-noise setting, we compare the underestimated allowance $\varepsilon_f = 10^{-4}$, the nominal allowance $\varepsilon_f = 10^{-2}$, and the overestimated allowance $\varepsilon_f = 10^{-1}$.
- *Floating-point-precision experiments* (results in section 5.2.4)
 - *64-bit arithmetic*: 220 problems with $\varepsilon_f = 2.22 \times 10^{-9}$.
 - *32-bit arithmetic*: 220 problems with $\varepsilon_f = 1.19 \times 10^{-3}$.
 - *16-bit arithmetic*: 152 problems with $\varepsilon_f = 9.77 \times 10^{-2}$.

Here, $\text{unif}(a, b)$ denotes a uniform random variable in the interval $[a, b]$. The additive uniform-noise model follows the model used by Shi et al. [36]. Each artificial-noise setting uses the five seeds 0, 1, 2, 3, 4. For NTQN in the function-value-noise and joint-noise experiments, we supplied the conservative absolute noise bound $\text{eps}_f = 10^{-2}$ because each function perturbation is bounded in magnitude by 10^{-3} .

In the reduced precision settings, to simulate lower numerical precision, we first cast the input vector x to the target lower precision, then computed $\bar{f}(x)$ and $\nabla \bar{f}(x)$ using the lower-precision input. Although the internal computations of CUTEst remain in 64-bit precision, the casting of the input vector effectively simulated the impact of reduced precision on function and gradient evaluations.

The error allowances for the three precision settings are summarized in Table 1. The 64-bit value follows SciPy’s default heuristic of $10^7 \varepsilon_{\text{mach}}$. For 32- and 16-bit arithmetic, we use the smaller multipliers 10^4 and 10^2 , respectively, because $10^7 \varepsilon_{\text{mach}}$ would violate $\varepsilon_f < 1$ and provide no informative allowance. These values are practical safety margins for accumulated floating-point errors rather than estimates supplied by the theory. Because the appropriate value of ε_f may not be known accurately in practice, the sensitivity experiment examines how under- and overestimation affect performance.

The function-value-noise setting satisfies Assumption 1, whereas the joint-noise setting deliberately violates it and serves only as a stress test beyond the theory. The latter setting is particularly adverse for our method because its analysis and regularization logic rely on accurate gradients, whereas NTQN was designed to accommodate bounded gradient errors. We also set the maximum number of iterations to $k_{\text{max}} = 15,000$ and timed out runs exceeding 10 minutes per problem.

5.2.2 Performance Profiles

We used performance profiles [10] to compare the algorithms. Let P denote the set of test instances and S the set of solvers. For each instance $p \in P$ and solver $s \in S$, let $n_{p,s} > 0$ denote the number of oracle calls (i.e., function and gradient evaluations)

explanation	min abs value	relative error	ϵ_f
64-bit double precision	2.23×10^{-308}	$2^{-52} \approx 2.22 \times 10^{-16}$	2.22×10^{-9}
32-bit single precision	1.18×10^{-38}	$2^{-23} \approx 1.19 \times 10^{-7}$	1.19×10^{-3}
16-bit half precision	6.10×10^{-5}	$2^{-10} \approx 9.77 \times 10^{-4}$	9.77×10^{-2}

Table 1: The “min abs value” is the smallest positive normalized number, the “relative error” is the maximum relative rounding error, and ϵ_f is the parameter in (1) used in our experiments.

required to reach a specified gradient norm tolerance ϵ_{gtol} . In this experiment, we use the number of oracle calls rather than clock time to avoid the influence of algorithmic overhead and to focus on the intrinsic optimization efficiency. When a solver fails to converge within the gradient tolerance or exceeds a maximum number of oracle calls, we set $n_{p,s} = +\infty$. Then, the performance ratio $r_{p,s}$ is defined as

$$r_{p,s} = \frac{n_{p,s}}{\min_{s' \in S} n_{p,s'}}.$$

For a given threshold $\tau \geq 1$ and solver s , the performance profile plots the proportion of **test instances** for which $r_{p,s} \leq \tau$. Thus, the y-coordinate for solver s at x-coordinate τ is given by

$$\rho_s(\tau) = \frac{1}{|P|} |\{p \in P \mid r_{p,s} \leq \tau\}|.$$

A curve that lies above others indicates better performance. Following standard implementations [40], we used the infinity norm for the gradient, though other norms may equally be employed. **All reported success percentages use the same common gradient tolerance as the corresponding profiles. A run that terminates without attaining that tolerance is counted as unsolved for this metric.**

5.2.3 Results with Artificial Noise

Table 2 summarizes the function-value noise experiment and the joint noise experiment described in section 5.2.1, and Figures 1 and 2 shows their performance profiles. The former satisfies the assumptions of section 3, whereas the latter tests empirical robustness beyond the exact-gradient theory.

Both analyzed variants solve at least one instance of dimension 123,200, the largest dimension in the retained collection. In the theory-aligned function-value-noise setting, Ours-R solves 87.6% of the instances, compared with 85.1% for Ours, and reduces the conditional median from 62 to 60 oracle calls. The joint-noise results suggest empirical robustness beyond the theory, but they do not establish gradient-noise tolerance or extend the exact-gradient result. Under NTQN’s recommended termination mode, 50.1% of the same function-noise instances attain the common tolerance, compared with 81.2% when the external common-tolerance stop is used. All recommended-mode runs terminate without timeout or exception, and 619 of

Method	Function-value noise		Joint noise	
	Solved (%)	Calls	Solved (%)	Calls
Ours	85.1 ± 0.67	62	83.6 ± 0.72	63
Ours-R	87.6 ± 0.81	60	–	–
Ours-MS	85.5 ± 0.20	67	84.8 ± 0.25	71
Ours-SP	83.9 ± 0.52	62	80.6 ± 0.69	62
NTQN	81.2 ± 1.49	64	71.9 ± 0.38	62
Reg	77.8 ± 1.13	60	73.5 ± 1.35	56
Reg-Sec	74.0 ± 0.87	54	73.0 ± 1.35	62
ASTR1-Adagrad	41.5 ± 1.26	131	42.8 ± 0.93	150

Table 2: Success rates and conditional median oracle calls at $\epsilon_{\text{gtol}} = 10^{-2}$ in the additive-noise experiments. Each setting contains 1,100 problem-seed instances. Success rates are reported as the mean $\pm$ sample standard deviation over the five seeds, in percentage points, and oracle-call medians are computed over successful runs.

them stop because NTQN detects that function-value changes have reached its estimated function-noise level (flag 4). The lower scored fraction therefore mainly reflects NTQN’s noise-dependent stopping rule rather than failed execution.

Figure 3 shows the performance profiles for the error-bound sensitivity experiment. The sensitivity results show that underestimation improves Ours but degrades Ours-MS, whereas overestimation lowers the solved percentage of both variants. Because underestimation can violate the assumed error bound, the improvement for Ours should not be interpreted as evidence that a smaller allowance is generally preferable. The optional restart curves permit a direct comparison with this heuristic.

5.2.4 Results at Different Precision Levels

We next present performance profiles for each floating-point precision (64-, 32-, and 16-bit). We evaluated each method at three gradient norm tolerance levels, i.e., $\epsilon_{\text{gtol}} \in \{10^{-1}, 10^{-3}, 10^{-5}\}$. These tolerance levels represent different optimization regimes, from moderate accuracy to high precision requirements. Figure 4 shows the comprehensive performance profiles across all three precision levels. At the strictest tolerance $\epsilon_{\text{gtol}} = 10^{-5}$, Ours and Ours-MS solve 84.5% and 85.5% of the 64-bit problems, 52.3% and 52.7% of the 32-bit problems, and 28.9% and 28.9% of the 16-bit problems. The decreasing rates quantify the expected loss of attainable accuracy as arithmetic precision is reduced, while both variants remain among the most robust methods in the profiles. Their relative ranking depends on the tolerance and precision, so we avoid claiming uniform superiority and report both the fraction solved and the performance ratios in Figure 4.

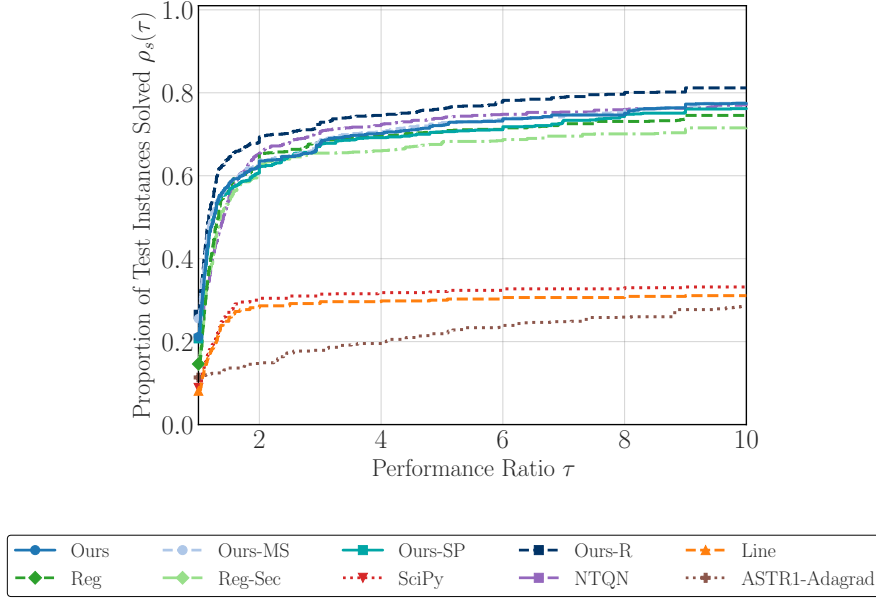


Fig. 1: Theory-aligned function-value-noise results at noise level 10^{-3} , supplied error bound $\epsilon_f = 10^{-2}$, and $\epsilon_{\text{gtol}} = 10^{-2}$.

5.3 Computational Time per Iteration

In this subsection, we present an experiment comparing the per-iteration computational cost of each method. Here, we measure total execution time rather than the number of oracle calls. We conducted experiments on the ill-conditioned quadratic problem

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad \frac{1}{2} \sum_{i=1}^n i x_i^2$$

with $n = 10,000$. The initial point x_0 was chosen as the all-ones vector. We used memory size $p = 10$ and maximum iterations $k_{\max} = 100$, and we did not set any stopping criteria other than reaching $k_{\max}$. The simplicity of the optimization problem and the small value of $k_{\max}$ are well suited to the purpose of this experiment, as they ensure that the total runtime is dominated by algorithmic overhead rather than function or gradient computation. Timing data were collected after three warm-up runs, followed by 100 recorded trials.

In Figure 5, we report the execution time statistics: the shifted-pair Ours-SP has per-iteration cost close to the least expensive Python implementations, whereas the exact compact solves in Ours and Ours-MS take about 1.4 times its runtime in this isolated test. This timing experiment measures algorithmic overhead under deliberately inexpensive objective and gradient evaluations and does not by itself establish end-to-end robustness or superiority in oracle cost. Because wall-clock time depends

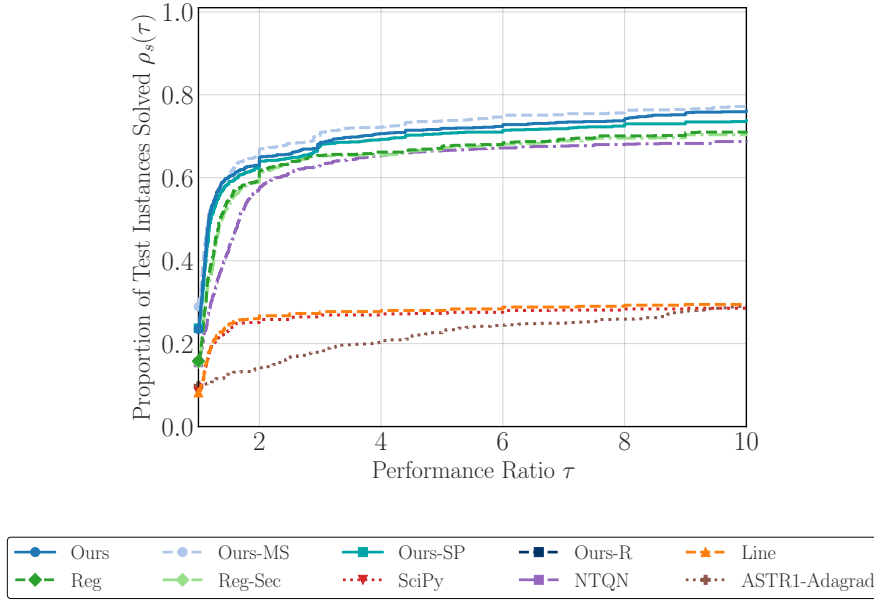


Fig. 2: Performance profile for the deliberately adverse joint function-and-gradient noise setting (componentwise level 10^{-3}) and $\epsilon_{\text{gtol}} = 10^{-2}$. This experiment lies outside the exact-gradient theory.

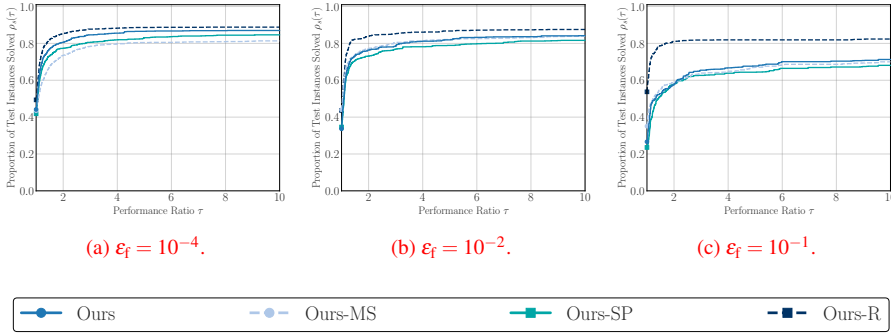


Fig. 3: Performance profiles for error-bound misspecification under function-value-only noise of level 10^{-3} and $\epsilon_{\text{gtol}} = 10^{-2}$.

strongly on implementation details and the computing environment, we restrict its interpretation to this isolated experiment and do not draw broader conclusions from the timing results. For a fairer comparison of algorithmic efficiency, we place greater emphasis on oracle-call counts and success profiles.

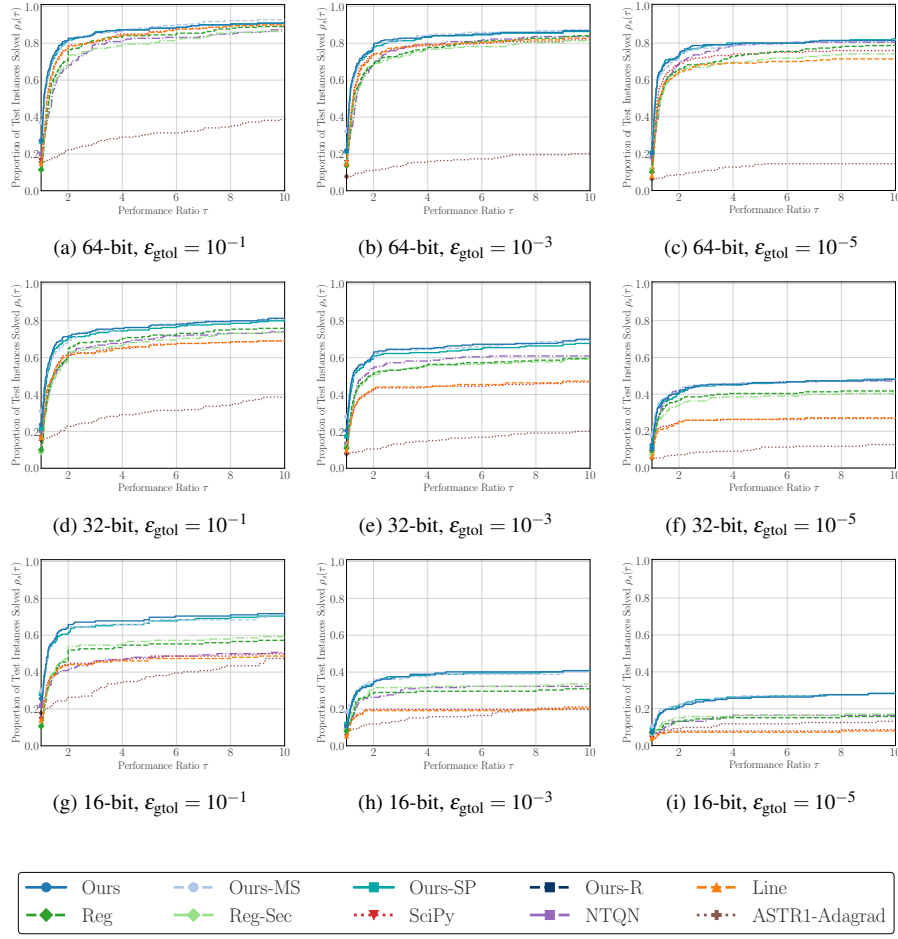


Fig. 4: Performance profiles across floating-point precisions (64-, 32-, and 16-bit) and tolerance levels.

In Table 3, we report the total number of oracle calls (calls of $\bar{f}$ and g) and final objective values $\bar{f}(x_{k_{\max}})$. To further check that the timing comparison is not driven only by a small fixed overhead or by fluctuations in very short runs, we report the 64-bit performance profile at $\epsilon_{\text{gtol}} = 10^{-3}$ separately for three runtime ranges. In Figure 6, problem instances are grouped according to the best runtime attained by any method: less than 1 s, between 1 and 10 s, and at least 10 s. The three panels show the same overall tendency as the existing experiments and indicate that the reported per-iteration timing is not merely an artifact of short-run overhead. The other parameter settings exhibit qualitatively similar behavior.

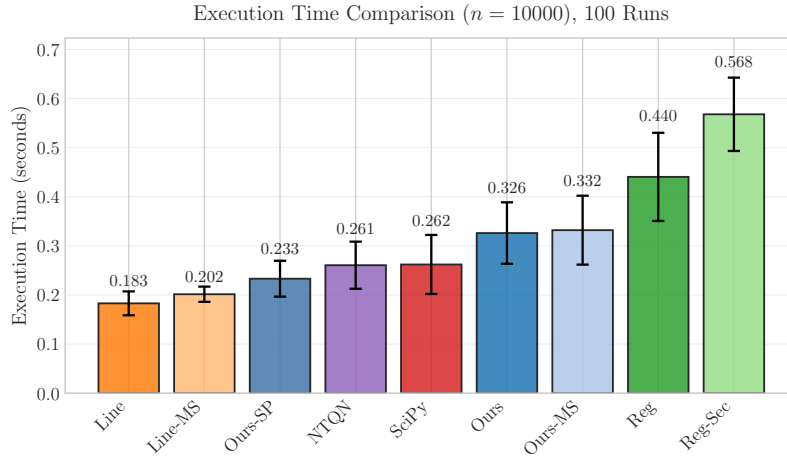


Fig. 5: Mean execution times for the experiment in section 5.3. This plot roughly indicates the extra computational overhead of each method. Error bars represent one standard deviation over 100 runs.

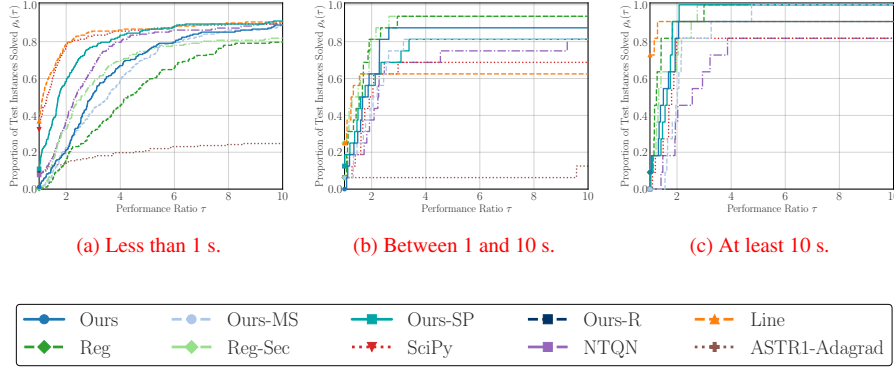


Fig. 6: Performance profiles for the 64-bit setting at $\epsilon_{\text{tol}} = 10^{-3}$, grouped by the best runtime attained by any method on each problem instance.

The purpose of this table is to check whether the methods achieved similar convergence behavior within the limited number of iterations. Comparable values would indicate that differences in convergence behavior do not dominate the execution-time comparison.

6 Conclusion

	Line	Ours-SP	NTQN	SciPy	Ours	Ours-MS	Reg	Reg-Sec
# Calls	212	202	219	212	202	202	206	206
$\bar{f}(x_{k_{\max}})$	1.24	1.34	1.43	1.24	1.34	1.34	1.47	1.48

Table 3: The number of oracle calls and final observed objective values for the experiment in section 5.3.

We proposed a hybrid regularized quasi-Newton method for smooth nonconvex optimization with bounded errors in objective-function values. The method retains the order-optimal $\mathcal{O}(1/\varepsilon^2)$ first-order iteration complexity of the L -smooth nonconvex setting while selectively using noisy function values. On the tested CUTEst problems, the proposed variants attained the common gradient tolerances across the reported function-noise, joint-noise, and reduced-precision settings. These results provide empirical evidence for the method under the stated benchmark protocol.

Future work includes the investigation of local convergence properties, the extension to constrained optimization, and the applications to practical problems in machine learning and scientific computing. Additionally, exploring adaptive strategies for selecting algorithmic parameters based on problem characteristics could further enhance the method’s performance. An explicit analysis of the effect of inexact gradient evaluations would also provide valuable insight into the method’s robustness and guide further enhancements.

References

1. Babaie-Kafaki S, Ghanbari R, Mahdavi-Amiri N (2010) Two new conjugate gradient methods based on modified secant equations. *Journal of Computational and Applied Mathematics* 234(5):1374–1386, DOI 10.1016/j.cam.2010.01.052
2. Bellavia S, Gurioli G, Morini B, Toint PL (2023) The Impact of Noise on Evaluation Complexity: The Deterministic Trust-Region Case. *Journal of Optimization Theory and Applications* 196(2):700–729, DOI 10.1007/s10957-022-02153-5
3. Berahas AS, Cao L, Choromanski K, Scheinberg K (2022) A Theoretical and Empirical Comparison of Gradient Approximations in Derivative-Free Optimization. *Foundations of Computational Mathematics* 22(2):507–560, DOI 10.1007/s10208-021-09513-z
4. Berahas AS, Curtis FE, Zhou B (2022) Limited-memory BFGS with displacement aggregation. *Mathematical Programming* 194(1):121–157, DOI 10.1007/s10107-021-01621-6
5. Carmon Y, Duchi JC, Hinder O, Sidford A (2020) Lower bounds for finding stationary points I. *Mathematical Programming* 184(1):71–120, DOI 10.1007/s10107-019-01406-y
6. Carter RG (1993) Numerical Experience with a Class of Algorithms for Nonlinear Optimization Using Inexact Function and Gradient Information. *SIAM Journal on Scientific Computing* 14(2):368–388, DOI 10.1137/0914023

7. Clancy RJ, Menickelly M, Hückelheim J, Hovland P, Nalluri P, Gjini R (2022) TROPHY: Trust Region Optimization Using a Precision Hierarchy. In: Computational Science – ICCS 2022, Springer, Cham, pp 445–459, DOI 10.1007/978-3-031-08751-6_32
8. Corless RM, Gonnet GH, Hare DEG, Jeffrey DJ, Knuth DE (1996) On the Lambert W function. *Advances in Computational Mathematics* 5(1):329–359, DOI 10.1007/BF02124750
9. Doikov N, Mishchenko K, Nesterov Y (2024) Super-Universal Regularized Newton Method. *SIAM Journal on Optimization* 34(1):27–56, DOI 10.1137/22M1519444
10. Dolan ED, Moré JJ (2002) Benchmarking optimization software with performance profiles. *Mathematical Programming* 91(2):201–213, DOI 10.1007/s101070100263
11. Duchi J, Hazan E, Singer Y (2011) Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *Journal of Machine Learning Research* 12(61):2121–2159, URL <http://jmlr.org/papers/v12/duchi11a.html>
12. Fowkes J, Roberts L, Bűrmen Á (2022) PyCUTEst: An open source Python package of optimization test problems. *Journal of Open Source Software* 7(78):4377, DOI 10.21105/joss.04377
13. Gao W, Goldfarb D (2018) Quasi-Newton Methods: Superlinear Convergence Without Line Searches for Self-Concordant Functions. *arXiv preprint arXiv:161206965* DOI 10.48550/arXiv.1612.06965, [1612.06965](https://arxiv.org/abs/1612.06965)
14. Gould NI, Orban D, Toint PL (2015) CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization. *Comput Optim Appl* 60(3):545–557, DOI 10.1007/s10589-014-9687-3
15. Gower RM, Goldfarb D, Richtárik P (2016) Stochastic Block BFGS: Squeezing More Curvature out of Data. *arXiv preprint arXiv:160309649* DOI 10.48550/arXiv.1603.09649, [1603.09649](https://arxiv.org/abs/1603.09649)
16. Grapiglia GN, Stella GFD (2022) An adaptive trust-region method without function evaluations. *Computational Optimization and Applications* 82(1):31–60, DOI 10.1007/s10589-022-00356-0
17. Gratton S, Toint PL (2020) A note on solving nonlinear optimization problems in variable precision. *Computational Optimization and Applications* 76(3):917–933, DOI 10.1007/s10589-020-00190-2
18. Gratton S, Toint PL (2023) OFFO minimization algorithms for second-order optimality and their complexity. *Computational Optimization and Applications* 84(2):573–607, DOI 10.1007/s10589-022-00435-2
19. Gratton S, Jerad S, Toint PhL (2026) Complexity of a class of first-order objective-function-free optimization algorithms. *Optimization Methods and Software* 41(2):478–508, DOI 10.1080/10556788.2023.2296431, <https://doi.org/10.1080/10556788.2023.2296431>
20. Hassan BA, Moghrabi IAR (2023) A modified secant equation quasi-Newton method for unconstrained optimization. *Journal of Applied Mathematics and Computing* 69(1):451–464, DOI 10.1007/s12190-022-01750-x
21. Higham NJ (2002) Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, DOI 10.1137/1.9780898718027

22. Izycheva A, Darulova E (2017) On sound relative error bounds for floating-point arithmetic. In: Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design, FMCAD Inc, Austin, Texas
23. Kanzow C, Steck D (2023) Regularization of limited memory quasi-Newton methods for large-scale nonconvex minimization. *Mathematical Programming Computation* 15(3):417–444, DOI 10.1007/s12532-023-00238-4
24. Li DH, Fukushima M (2001) On the Global Convergence of the BFGS Method for Nonconvex Unconstrained Optimization Problems. *SIAM Journal on Optimization* 11(4):1054–1064, DOI 10.1137/S1052623499354242
25. Li YJ, Li DH (2009) Truncated regularized Newton method for convex minimizations. *Computational Optimization and Applications* 43(1):119–131, DOI 10.1007/s10589-007-9128-7
26. Liu DC, Nocedal J (1989) On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45(1):503–528, DOI 10.1007/BF01589116
27. Mannel F (2025) A Globalization of L-BFGS and the Barzilai–Borwein Method for Nonconvex Unconstrained Optimization. *Journal of Optimization Theory and Applications* 204(3):1–34, DOI 10.1007/s10957-024-02565-5
28. Mannel F, Om Aggrawal H, Modersitzki J (2024) A structured L-BFGS method and its application to inverse problems. *Inverse Problems* 40(4):045022, DOI 10.1088/1361-6420/ad2c31
29. Moré JJ, Thuente DJ (1994) Line search algorithms with guaranteed sufficient decrease. *ACM Trans Math Softw* 20(3):286–307, DOI 10.1145/192115.192132
30. Nesterov Y, Polyak B (2006) Cubic regularization of Newton method and its global performance. *Mathematical Programming* 108(1):177–205, DOI 10.1007/s10107-006-0706-8
31. Nocedal J, Wright SJ (1999) *Numerical Optimization*. Springer, DOI 10.1007/978-0-387-40065-5
32. Okazaki N (2007) libLBFGS: A library of limited-memory broyden-fletcher-goldfarb-shanno (l-BFGS). URL <https://github.com/chokkan/liblbfgs>
33. Ranganath A, Singhal M, Marcia R (2025) Symmetric Rank-One Quasi-Newton Methods for Deep Learning Using Cubic Regularization. In: 2025 IEEE 10th International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP), pp 121–125, DOI 10.1109/CAMSAP66162.2025.11423871
34. Ruan H, Yang W (2024) Adaptive Regularized Quasi-Newton Method Using Inexact First-Order Information. *Journal of Computational Mathematics* 42(6):1656–1687, DOI 10.4208/jcm.2306-m2022-0279
35. Sheng Z, Yuan G, Cui Z (2018) A new adaptive trust region algorithm for optimization problems. *Acta Mathematica Scientia* 38(2):479–496, DOI 10.1016/S0252-9602(18)30762-8
36. Shi HJM, Xie Y, Byrd R, Nocedal J (2022) A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization. *SIAM Journal on Optimization* 32(1):29–55, DOI 10.1137/20M1373190
37. Sun S, Nocedal J (2023) A trust region method for noisy unconstrained optimization. *Mathematical Programming* 202(1):445–472, DOI 10.1007/

- s10107-023-01941-9
38. Ueda K, Yamashita N (2010) Convergence Properties of the Regularized Newton Method for the Unconstrained Nonconvex Optimization. *Applied Mathematics and Optimization* 62(1):27–46, DOI 10.1007/s00245-009-9094-9
 39. Ueda K, Yamashita N (2014) A regularized Newton method without line search for unconstrained optimization. *Computational Optimization and Applications* 59(1):321–351, DOI 10.1007/s10589-014-9656-x
 40. Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, Weckesser W, Bright J, van der Walt SJ, Brett M, Wilson J, Millman KJ, Mayorov N, Nelson ARJ, Jones E, Kern R, Larson E, Carey CJ, Polat I, Feng Y, Moore EW, VanderPlas J, Laxalde D, Perktold J, Cimrman R, Henriksen I, Quintero EA, Harris CR, Archibald AM, Ribeiro AH, Pedregosa F, van Mulbregt P, SciPy 10 Contributors (2020) SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods* 17:261–272, DOI 10.1038/s41592-019-0686-2
 41. Wang S, Fadili J, Ochs P (2025) Convergence rates of regularized quasi-Newton methods without strong convexity. *arXiv preprint arXiv:250600521* DOI 10.48550/arXiv.2506.00521, [2506.00521](https://arxiv.org/abs/2506.00521)
 42. Ward R, Wu X, Bottou L (2020) AdaGrad stepsizes: Sharp convergence over nonconvex landscapes. *Journal of Machine Learning Research* 21(219):1–30, URL <http://jmlr.org/papers/v21/18-352.html>
 43. Xie Y, Byrd RH, Nocedal J (2020) Analysis of the BFGS Method with Errors. *SIAM Journal on Optimization* 30(1):182–209, DOI 10.1137/19M1240794
 44. Yabe H, Ogasawara H, Yoshino M (2007) Local and superlinear convergence of quasi-Newton methods based on modified secant conditions. *Journal of Computational and Applied Mathematics* 205(1):617–632, DOI 10.1016/j.cam.2006.05.018
 45. Yuan G, Wei Z (2010) Convergence analysis of a modified BFGS method on convex minimizations. *Computational Optimization and Applications* 47(2):237–255, DOI 10.1007/s10589-008-9219-0
 46. Zhang H, Ni Q (2018) A new regularized quasi-Newton method for unconstrained optimization. *Optimization Letters* 12(7):1639–1658, DOI 10.1007/s11590-018-1236-z
 47. Zhang J, Xu C (2001) Properties and numerical performance of quasi-Newton methods with modified quasi-Newton equations. *Journal of Computational and Applied Mathematics* 137(2):269–278, DOI 10.1016/S0377-0427(00)00713-5
 48. Zhang JZ, Deng NY, Chen LH (1999) New Quasi-Newton Equation and Related Methods for Unconstrained Optimization. *Journal of Optimization Theory and Applications* 102(1):147–167, DOI 10.1023/A:1021898630001